Universidad de San Carlos de Guatemala
Facultad de Ingeniería
LFP

Manual de Tecnico - MedLexer

Nombre: Julio Eduardo Gustavo Alvarado Ramirez
Carnet: 202307764
Sección: A-

Introducción

El presente documento detalla la arquitectura, diseño y funcionamiento interno del proyecto "MedLexer", un analizador léxico desarrollado en C++17 con interfaz gráfica en Qt 6. El sistema está diseñado para leer, tokenizar y validar archivos de entrada escritos en el lenguaje específico "MedLang", extrayendo información estructurada sobre pacientes, médicos, citas y diagnósticos, para posteriormente generar reportes estadísticos en formato HTML y diagramas de red mediante Graphviz.

Requisitos del Entorno (Herramientas utilizadas)

- Lenguaje base: C++ (Estándar C++17)
- Framework Gráfico: Qt 6.11.0 (Componente: Widgets)
- Compilador: GNU MinGW 13.1.0 64-bit (x86_64-w64-mingw32)
- Gestor de Construcción: CMake 3.30.5
- Herramientas Externas: Graphviz (dot) para la renderización de imágenes PNG.

Arquitectura del Sistema (Diagrama de Clases)

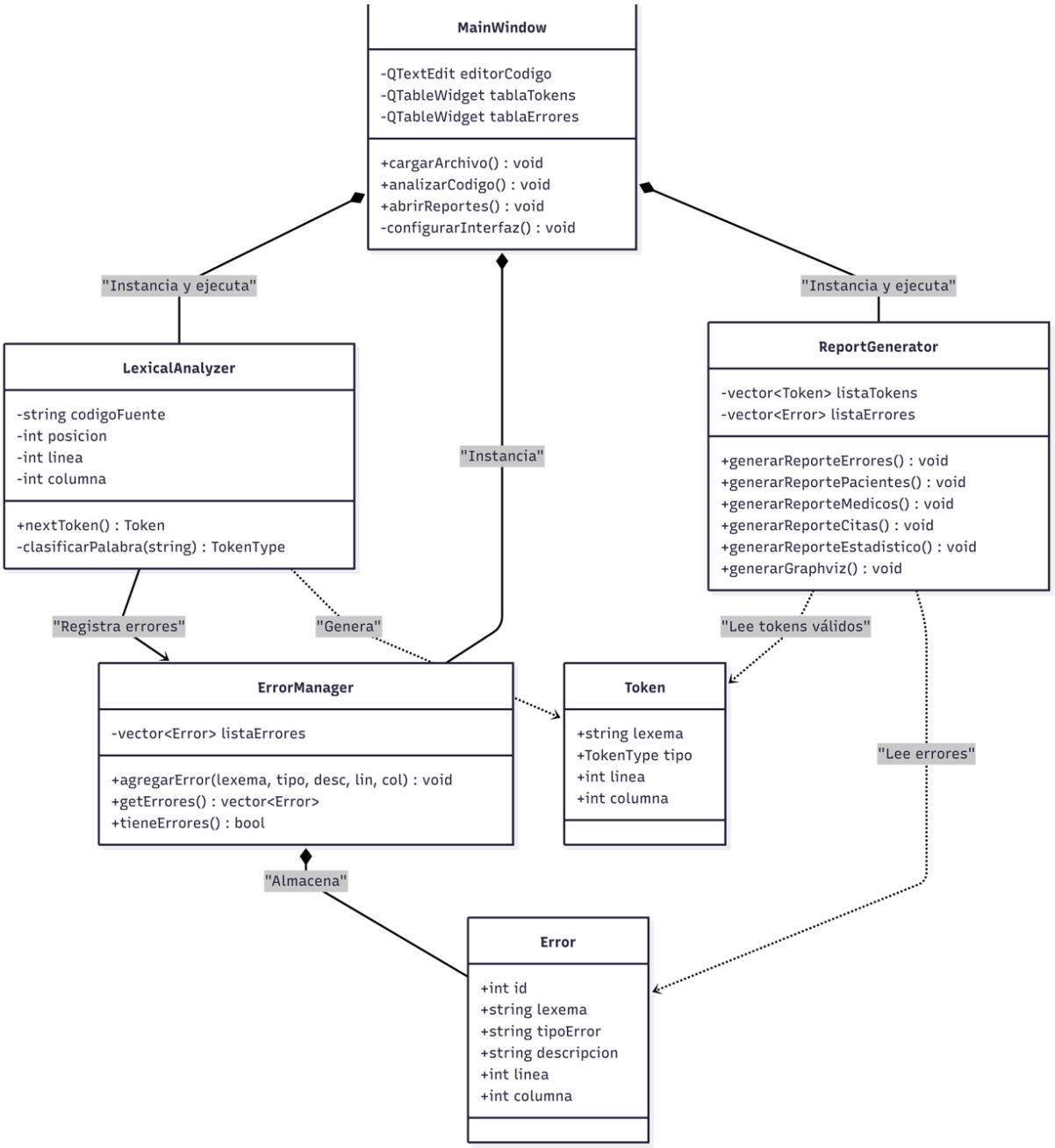
El sistema fue construido utilizando el principio de responsabilidad única, separando la lógica de la interfaz gráfica del procesamiento de datos.

Descripción de Clases Principales:

MainWindow: Actúa como el controlador principal de la GUI. Gestiona los eventos de los botones y actualiza las tablas visuales (QTableWidget) sin mezclarse con la lógica de tokenización.
LexicalAnalyzer: Es el núcleo del proyecto. Implementa el Autómata Finito Determinista (AFD) leyendo el código fuente caracter por caracter para agruparlos en estructuras de tipo Token.
ErrorManager: Clase dedicada exclusivamente a la recolección de caracteres inválidos. Si el analizador encuentra un símbolo fuera del

alfabeto de MedLang, se delega a esta clase para su registro sin detener el programa.
ReportGenerator: Toma los vectores de Tokens válidos y Errores Léxicos para estructurar archivos de salida interactivos (HTML) y el archivo de sintaxis .dot de Graphviz.

Diagramas
El analizador léxico reconoce los siguientes tipos de tokens (definidos en la enumeración TokenType): Palabras Reservadas, Identificadores, Cadenas (entre comillas), Números y Símbolos especiales ({, }, [,], :, ,).

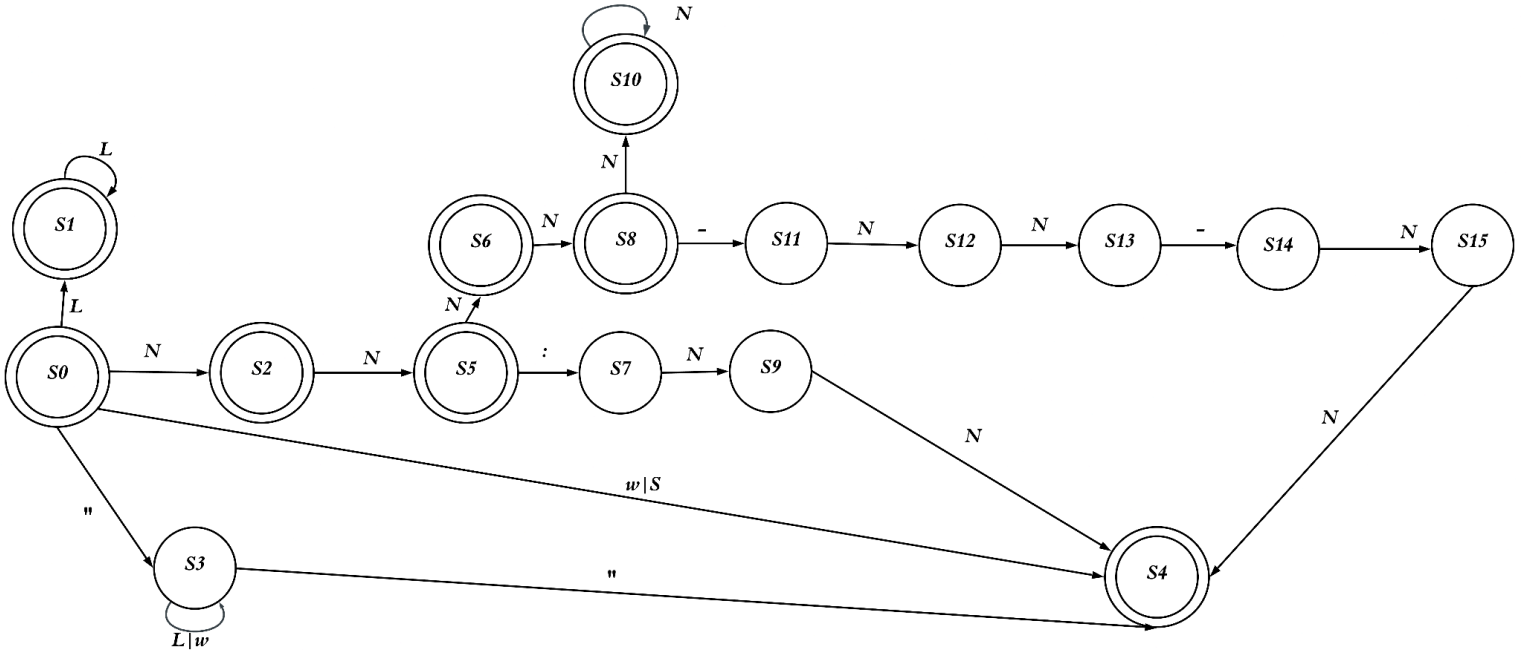


Análisis Léxico y Autómata (AFD)

El analizador léxico reconoce los siguientes tipos de tokens (definidos en la enumeración TokenType): Palabras Reservadas, Identificadores, Cadenas (entre comillas), Números y Símbolos especiales (**{, }, [,], :, ,**).

	Symbol	Follow Pos
1	L	1, 24
2	N	2, 24
3	"	4, 5, 6
4	L	4, 5, 6
5	w	4, 5, 6
6	"	24
7	S	24
8	w	24
9	N	10
10	N	11
11	N	12
12	N	13
13	-	14
14	N	15
15	N	16
16	-	17
17	N	18
18	N	24
19	N	20
20	N	21
21	:	22
22	N	23
23	N	24

	L	N	q	w	S	-	:
*S0	S1	S2	S3	S4	S4	-	-
*S1	S1	-	-	-	-	-	-
*S2	-	S5	-	-	-	-	-
S3	S3	-	S4	S3	-	-	-
*S4	-	-	-	-	-	-	-
*S5	-	S6	-	-	-	-	S7
*S6 }	-	S8	-	-	-	-	-
S7	-	S9	-	-	-	-	-
*S8	-	S10	-	-	-	S11	-
S9	-	S4	-	-	-	-	-
*S10	-	S10	-	-	-	-	-
S11	-	S12	-	-	-	-	-
S12	-	S13	-	-	-	-	-
S13	-	-	-	-	-	S14	-
S14	-	S15	-	-	-	-	-
S15	-	S4	-	-	-	-	-



Manejo y Recuperación de Errores (Estado S6):
Una de las características más robustas del sistema es su capacidad de recuperación ante errores léxicos. Si durante el Estado Inicial (S0) el autómata lee un carácter que no pertenece al alfabeto del lenguaje (por ejemplo, @, &, #), no lanza una excepción fatal. En su lugar, transita al estado S6, encapsula el carácter como ERROR_LEXICO, lo envía al ErrorManager, e incrementa los contadores de posición y columna para continuar el análisis desde el siguiente carácter.

```
#include "ErrorManager.h"

ErrorManager::ErrorManager() : contadorId(1) {}

void ErrorManager::agregarError(const std::string& lexema, const std::string& tipoError,
                                const std::string& descripcion, int linea, int columna,
                                const std::string& gravedad) {
    ErrorLexico nuevoError = {contadorId++, lexema, tipoError, descripcion, linea, columna, gravedad};
    listaErrores.push_back(nuevoError);
}

const std::vector<ErrorLexico>& ErrorManager::getErrores() const {
    return listaErrores;
}

bool ErrorManager::tieneErrores() const {
    return !listaErrores.empty();
}

void ErrorManager::limpiar() {
    listaErrores.clear();
    contadorId = 1;
}
```

Estructuras de Datos
Para el almacenamiento temporal de la información durante el tiempo de ejecución (runtime), se optó por el uso de la Standard Template Library (STL) de C++.

Vectores Dinámicos (std::vector): Se utilizaron std::vector<Token> y std::vector<Error> debido a su eficiencia para redimensionarse automáticamente conforme se leen archivos de gran tamaño, permitiendo un acceso secuencial rápido durante la generación de reportes.

```cpp
#pragma once
#include <string>

enum class TokenType {
    HOSPITAL, PACIENTES, MEDICOS, CITAS, DIAGNOSTICOS,
    PACIENTE_ELEM, MEDICO_ELEM, CITA_ELEM, DIAGNOSTICO_ELEM,
    CADENA, NUMERO, FECHA, HORA, ID_CODIGO,
    ESPECIALIDAD, TIPO_SANGRE, DOSIS, SIMBOLO,
    ERROR_LEXICO, FIN_ARCHIVO
};

struct Token {
    std::string lexema;
    TokenType tipo;
    int linea;
    int columna;
};
```

Structs Personalizados: Se crearon las estructuras ligeras Token (almacena lexema, tipo, línea y columna) y Error (almacena id, lexema, tipo, descripción, línea y columna) para empaquetar la información de forma tipada.

```cpp
#pragma once
#include <string>
#include <vector>

// Estructura del error
struct ErrorLexico {
    int id;
    std::string lexema;
    std::string tipoError;
    std::string descripcion;
    int linea;
    int columna;
    std::string gravedad;
};

// Declaración de la clase
class ErrorManager {
private:
    std::vector<ErrorLexico> listaErrores;
    int contadorId;

public:
    ErrorManager();
    void agregarError(const std::string& lexema, const std::string& tipoError,
                      const std::string& descripcion, int linea, int columna,
                      const std::string& gravedad = "ERROR");
    const std::vector<ErrorLexico>& getErrores() const;
    bool tieneErrores() const;
    void limpiar();
};
```

Generación de Reportes y Gráficos

Una vez finalizado el análisis léxico, la clase ReportGenerator utiliza flujos de salida de archivos (std::ofstream) para escribir código HTML dinámico, iterando sobre los vectores de tokens para construir tablas estilizadas con CSS embebido.

Para el diagrama de la red de hospitales, el sistema genera primero un archivo hospital.dot con sintaxis de nodos y aristas. Posteriormente, mediante la librería <cstdlib>, se invoca una llamada directa al sistema operativo (std::system()) para ejecutar el motor de renderizado local de

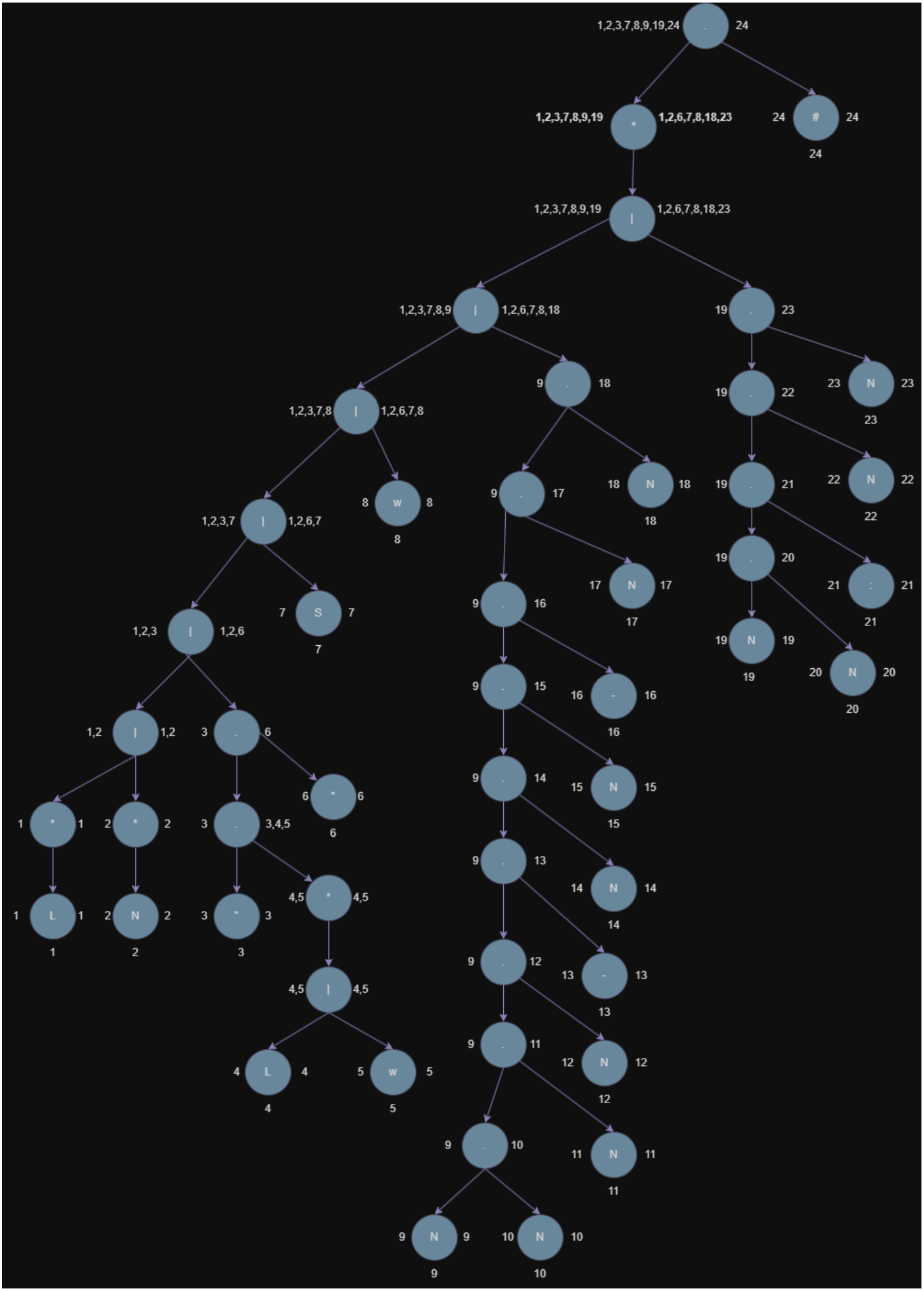
```
ReportGenerator::ReportGenerator(const std::vector<Token>& tokens, const std::vector<ErrorLexico>& errores)
    : listaTokens(tokens), listaErrores(errores) {}

// --- MÉTODOS DE AYUDA PARA HTML (CSS Embebido) ---
std::string ReportGenerator::obtenerEncabezadoHTML(const std::string& titulo) {
    // Un diseño limpio con CSS embebido como pide el proyecto
    return "<!DOCTYPE html>\n<html>\n<head>\n<meta charset=\"UTF-8\">\n<title>" + titulo + "</title>\n"
           "<style>\n"
           "body { font-family: Arial, sans-serif; background-color: #f4f7f6; margin: 20px; }\n"
           "h1 { color: #2c3e50; text-align: center; }\n"
           "table { width: 100%; border-collapse: collapse; margin-top: 20px; background-color: #fff; }\n"
           "th, td { border: 1px solid #ddd; padding: 12px; text-align: left; }\n"
           "th { background-color: #34495e; color: white; }\n"
           "tr:nth-child(even) { background-color: #f2f2f2; }\n"
           ".error { color: #e74c3c; font-weight: bold; }\n"
           "</style>\n</head>\n<body>\n"
           "<h1>" + titulo + "</h1>\n";
}

std::string ReportGenerator::obtenerPieHTML() {
    return "</body>\n</html>";
}
```

Fundamento Teórico: Método del Árbol

Para garantizar que el Autómata Finito Determinista (AFD) programado en la clase LexicalAnalyzer sea exacto y no contenga ambigüedades, su diseño lógico fue respaldado por el Método del Árbol Sintáctico a partir de las expresiones regulares que componen el lenguaje MedLang.


1,2,3,7,8,9,19,24 . 24
1,2,3,7,8,9,19 * 1,2,6,7,8,18,23
24 # 24
24
1,2,3,7,8,9,19 | 1,2,6,7,8,18,23
1,2,3,7,8,9 | 1,2,6,7,8,18
19 . 23
1,2,3,7,8 | 1,2,6,7,8
9 . 18
19 . 22
23 N 23
23
1,2,3,7 | 1,2,6,7
8 w 8
8
9 . 17
18 N 18
18
19 . 21
22 N 22
22
1,2,3 | 1,2,6
7 S 7
7
17 N 17
17
19 . 20
21 : 21
21
19 N 19
19
20 N 20
20
1,2 | 1,2
3 . 6
1 * 1
2 * 2
3 . 3,4,5
6 " 6
6
1 L 1
1
2 N 2
2
3 " 3
3
4,5 * 4,5
4,5 | 4,5
4 L 4
4
5 w 5
5
9 . 16
16 - 16
16
9 . 15
15 N 15
15
9 . 14
14 N 14
14
9 . 13
13 - 13
13
9 . 12
12 N 12
12
9 . 11
11 N 11
11
9 . 10
9 N 9
9
10 N 10
10

Descripción del proceso:
El diagrama ilustra la jerarquización de las expresiones regulares del lenguaje mediante operaciones de concatenación (.), alternancia (|) y cerradura de Kleene (* o +). A partir de esta estructura, se determinaron teóricamente los estados del autómata mediante el cálculo de anulabilidad y las funciones FirstPos y LastPos para cada nodo. Posteriormente, las hojas del árbol (que representan el alfabeto y los símbolos del lenguaje) permitieron calcular la Tabla de Siguientes (FollowPos). Esta tabla fue crucial, ya que dictó matemáticamente las reglas de transición: es decir, hacia qué estado debe saltar el autómata al leer un caracter específico.
Implementación en el Código:
El resultado final de este análisis de FollowPos es precisamente lo que se mapeó en el código fuente de C++ dentro del método nextToken(). Las transiciones que se observan en el árbol y en su tabla resultante dictan el comportamiento de los ciclos y las condiciones que validan los Identificadores, Cadenas y Símbolos de MedLang, asegurando un análisis léxico libre de errores de estado.